

Object-Oriented Programming

Final Term Project

OOP Analysis of NumPy

A Study of Object-Oriented Design in a Real Python Library

Submitted by:

Muhammad Ahsan Khawar 02091

Tooba Saeed 02054

Talia Tahir 02079

BS Data Science | Semester 2 | Spring 2026

Submission Date: 31 May 2026 | Total Marks: 100

1. Library Overview

For this project we chose NumPy, which stands for Numerical Python. NumPy is a free and open-source Python library that was first released in 2005. Its main purpose is to give programmers a way to work with large collections of numbers arranged in grids, which are called arrays. Before NumPy existed, doing heavy math work in Python was very slow because regular Python lists are not made for that kind of work. NumPy solved this by creating a new type of object called an ndarray, which stores numbers in a very efficient way so that math operations run at speeds close to languages like C and Fortran.

To install NumPy you just need to run one line in the terminal:

```
pip install numpy
```

After installing, the standard way to use it in any Python file is:

```
import numpy as np
```

We chose NumPy for this project because it is one of the most used libraries in Data Science and Machine Learning. Almost every other library we will use in our degree, like Pandas, Scikit-learn, and Matplotlib, all depend on NumPy underneath. More importantly for this project, NumPy is built with very clear Object-Oriented design. It shows all four OOP principles, which are encapsulation, inheritance, polymorphism, and abstraction, in real working code that millions of people use every day.

In real life, NumPy is used by data scientists to clean and process datasets, by machine learning engineers to do the matrix calculations inside neural networks, and by researchers to run simulations. Even images are stored as NumPy arrays. A photograph is just a three-dimensional array where the three dimensions represent height, width, and the three colour channels red, green, and blue.

2. Class Hierarchy and Structure

The first thing we did when studying NumPy was look at how its classes are organised. We found that the whole library is built around one central class called `ndarray`, which stands for N-dimensional array. Every array you create in NumPy is an object of this class. All the other important classes either inherit from `ndarray` or work closely beside it.

2.1 The `ndarray` Class

The `ndarray` class is the heart of NumPy. When you write `np.array([1, 2, 3])`, what you get back is an `ndarray` object. This object stores your data internally as a flat block of raw bytes in memory, but it hides all of that from you and gives you clean, easy-to-use attributes instead. The main attributes are:

- **shape** tells you the size of the array in each dimension. A 2D array with 3 rows and 4 columns would have shape (3, 4).
- **dtype** tells you what type of data is stored, like `int64` for whole numbers or `float64` for decimal numbers.
- **ndim** tells you how many dimensions the array has. A simple list is 1D, a table is 2D.
- **size** tells you the total number of elements. A (3, 4) array has size 12.
- **data** is the raw memory buffer where the bytes are stored. You almost never need to access this directly.
- **strides** tells NumPy how many bytes to jump in memory to get to the next element in each direction.

2.2 Subclasses of `ndarray`

Three important classes in NumPy inherit from `ndarray`. Each one takes everything `ndarray` already has and adds or changes something for a specific purpose.

The `matrix` class inherits from `ndarray` and overrides the multiplication operator. In a normal `ndarray`, the `*` symbol multiplies each element with the element at the same position in the other array. The `matrix` class changes this so that `*` does proper matrix multiplication instead, which is what you need when doing linear algebra. This is one of the clearest examples of inheritance and method overriding we found in the whole library.

The `memmap` class also inherits from `ndarray` and is designed for working with very large files that cannot fit entirely into RAM. Instead of loading all the data at once, it maps the file on disk to memory and only loads the parts you actually need at any given time. From the outside it looks and behaves exactly like a regular `ndarray`, which is the whole point of inheritance.

The `MaskedArray` class comes from the `numpy.ma` module and inherits from `ndarray`. It adds the ability to mark certain values in an array as missing or invalid. This is very useful in Data Science because real datasets almost always have some missing values. It stores a second array called a

mask alongside the data, and wherever the mask is True, that value is left out of calculations.

2.3 Related Classes

Alongside the inheritance chain there are a few other important classes. The dtype class describes the data type and memory layout of each element. The ufunc class, which stands for universal function, wraps math operations like np.add and np.sin so they work on whole arrays at once. The nditer class is an iterator that lets you loop through multi-dimensional arrays in a controlled and flexible way. These three do not inherit from ndarray but they are used by it all the time.

3. OOP Principles Found in NumPy

This was the most interesting part of the project for us. We went through the NumPy documentation and source code and looked for real examples of each OOP principle. Below is what we found, with working code examples to back each one up.

3.1 Encapsulation

Encapsulation means hiding the complicated internal details of an object and only showing a clean and simple interface to whoever is using it. The best example of this in NumPy is the ndarray class itself.

When you create a NumPy array, your data is stored internally as a flat sequence of raw bytes in a memory buffer. This is very low-level and not something any programmer would want to deal with directly. But NumPy hides all of that completely. Instead of showing you raw bytes, it gives you clean attributes like `.shape`, `.dtype`, `.ndim`, and `.size`. Here is a simple example:

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape) # (2, 3)
print(a.dtype) # int64
print(a.ndim) # 2
print(a.size) # 6
print(a.data) # <memory at 0x...> -- hidden raw bytes
```

The `.strides` attribute is another good example. It tells NumPy how many bytes to jump to get from one element to the next. When you write `a[1][2]` you do not need to know anything about byte offsets. NumPy uses strides internally to find the right location in memory. That is exactly what encapsulation is about.

3.2 Inheritance

Inheritance means one class can take everything from another class and then add or change certain things to suit a more specific purpose. The best example in NumPy is the matrix class, which inherits from ndarray and overrides the `*` operator.

```
a = np.array([[1, 2], [3, 4]])
m = np.matrix([[1, 2], [3, 4]])
print(a * a) # element-wise: [[ 1  4] [ 9 16]]
print(m * m) # matrix multiply: [[ 7 10] [15 22]]
```

Both arrays contain the same numbers. But the matrix class has overridden the `__mul__` method so that `*` means something different. The matrix class still gets all of ndarray's attributes and methods for free. It only changes the one thing that needs to be different. The memmap and MaskedArray classes work the same way.

3.3 Polymorphism

Polymorphism means the same method name or operator can behave in different ways depending on what kind of object it is used on. We found two very clear examples of this in NumPy.

The first example is operator overloading. The ndarray class overrides the standard Python + and * operators by defining special methods called `__add__` and `__mul__`. This means the + symbol does something completely different when used with NumPy arrays compared to Python lists:

```
my_list = [1, 2, 3]
print(my_list + my_list) # [1, 2, 3, 1, 2, 3] -- joins the lists
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # [5 7 9] -- adds element by element
```

The second example is the `.dot()` method. The same method produces two completely different results depending on whether you give it 1D arrays or 2D arrays:

```
# 1D arrays -- gives back a single number
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a.dot(b)) # 32 because (1*4)+(2*5)+(3*6) = 32

# 2D arrays -- gives back a full matrix
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A.dot(B)) # [[19 22] [43 50]]
```

Same method name, same syntax, but completely different output depending on the input. This is polymorphism working in practice.

3.4 Abstraction

Abstraction means hiding how something works on the inside and only showing what it does from the outside. NumPy is full of this. Methods like `.reshape()`, `.flatten()`, and `.astype()` each do quite complex things internally but present a very simple interface to the programmer.

```
a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape(2, 3) # [[1 2 3] [4 5 6]]
c = a.astype(float) # [1. 2. 3. 4. 5. 6.]
d = b.flatten() # [1 2 3 4 5 6]
```

When we call `a.reshape(2, 3)` we are not writing any code to rearrange memory or recalculate strides. We just say what shape we want and NumPy handles everything else. The internal complexity is completely hidden. This is one of the reasons NumPy is so easy to use despite being very powerful under the hood.

4. Design Decision: The Homogeneity Constraint

As part of this project we were asked to look at one major design decision made by the NumPy developers and think about its trade-offs. We chose to look at the decision that every element in an ndarray must be of the same data type. This is called the homogeneity constraint.

4.1 What the Decision Is

In NumPy you cannot have an array that mixes integers and strings, or even integers and floats as separate types. If you try to mix them, NumPy automatically converts all the values to the most general type. For example if you mix integers and floats, everything becomes float. If you mix numbers and text, everything becomes a string.

```
a = np.array([1, 2, 3.5])
print(a) # [1. 2. 3.5]
print(a.dtype) # float64

b = np.array([1, 2, 'hello'])
print(b) # ['1' '2' 'hello']
print(b.dtype) # string type
```

4.2 Why the Developers Made This Decision

The reason for this constraint is performance. When all elements in an array are the same type and the same size in bytes, NumPy can store them in one continuous block of memory with no gaps and no extra type information per element. Because of this layout, NumPy can hand the data directly to optimised C and Fortran code that runs at hardware speed. This is why NumPy operations can be 50 to 100 times faster than writing the same thing with Python loops and regular lists.

4.3 The Trade-offs

The performance gain is real and very significant, but the constraint does cause some problems. The most dangerous one is that type conversion happens silently. If a programmer accidentally puts a float into what they meant to be an integer array, NumPy will not raise an error. It will just quietly convert everything to float. This can introduce hard-to-find bugs in large data pipelines.

The second trade-off is that NumPy arrays are not good for datasets that mix different types in the same structure, like a table where one column is a name, another is an age, and another is a balance. Python lists handle this naturally but NumPy does not. This is exactly why Pandas was created on top of NumPy. Pandas lets you work with mixed-type tables while still using homogeneous NumPy arrays for each individual column internally. We think this is the right approach: keep NumPy fast and strict, and build something friendlier on top for when mixed types are needed.

5. Comparison with Python's Built-in array Module

Python actually comes with its own built-in array module that you can import as 'import array'. We chose to compare NumPy against this because on the surface they sound similar. Both store typed collections of numbers. But when you actually look at them side by side the difference is very large.

Python's `array.array` is a one-dimensional typed sequence. You can create an array of integers or doubles and it stores them more efficiently than a regular Python list. But that is about all it does. It has no concept of dimensions, no shape attribute, no mathematical operations, and no methods like `reshape` or `flatten`. Here is what happens when you try to use it like a NumPy array:

```
import array
pa = array.array('i', [1, 2, 3, 4, 5, 6])
# pa.reshape(2, 3) -- AttributeError, this method does not exist
# pa + pa -- TypeError, this does not work either
na = np.array([1, 2, 3, 4, 5, 6])
print(na.reshape(2, 3)) # works perfectly
print(na + na) # also works fine
```

From an OOP design point of view the two libraries are not even in the same category. Python's array module has one class with basic storage functionality and nothing else. There is no class hierarchy, no inheritance chain, no operator overloading for math, and no abstraction through powerful methods. NumPy on the other hand is built on a rich OOP foundation. The `ndarray` class demonstrates all four OOP principles clearly. For Data Science purposes the built-in array module is essentially never used. NumPy has been the standard for over fifteen years.

6. Our Custom Extension: SmartArray

For the coding part of this project we built a custom class called SmartArray that inherits from `numpy.ndarray`. We added three new methods that are genuinely useful for Data Science work, and we also overrode one existing method to show polymorphism in our own code. We split the implementation across multiple Python files, with each file focused on one OOP concept, to make it clear which part demonstrates which idea.

6.1 Setting Up Inheritance

Inheriting from `numpy.ndarray` is a little different from normal Python inheritance because `ndarray` is built at the C level. The standard `__init__` method does not work for it. Instead, NumPy requires you to override a method called `__new__`, which is responsible for actually creating the object in memory. There is also a method called `__array_finalize__` that NumPy calls automatically every time a new SmartArray is created. We have to define it even if we leave it empty.

```
import numpy as np

class SmartArray(np.ndarray):
    def __new__(cls, input_data):
        obj = np.asarray(input_data).view(cls)
        return obj
    def __array_finalize__(self, obj):
        pass
```

With just these two methods SmartArray already inherits the complete `ndarray` interface. Every attribute like `shape`, `dtype`, `ndim`, `size` and every method like `reshape`, `flatten`, `astype`, and `dot` works automatically without us writing a single line of code for any of them. That is the power of inheritance.

6.2 The `summary()` Method - Encapsulation

The `summary()` method shows encapsulation. It bundles together seven different pieces of information about the array and presents them in a single clean output. Instead of writing seven separate print statements every time, the user just calls `summary()` and everything appears at once.

```
def summary(self):
    print('=' * 40)
    print(' SmartArray Summary')
    print('=' * 40)
    print(' Shape :', self.shape)
    print(' Dimensions :', self.ndim)
    print(' Size :', self.size)
    print(' Data Type :', self.dtype)
    print(' Min Value :', int(self.min()))
    print(' Max Value :', int(self.max()))
    print(' Mean Value :', round(float(self.mean()), 2))
    print('=' * 40)
```

6.3 The normalize() Method - Abstraction

The `normalize()` method shows abstraction. Min-Max normalisation is a standard technique in Data Science that scales all values in a dataset to be between 0 and 1. It is used before feeding data into machine learning models because many algorithms work better when all numbers are on the same scale. The formula is: $\text{normalised value} = (\text{value} - \text{minimum}) / (\text{maximum} - \text{minimum})$. Our `normalize()` method completely hides this formula. The user does not need to know it, remember it, or worry about what happens when all values are the same, which would cause a division by zero. All of that is handled inside the method.

```
def normalize(self):
    mn = self.min()
    mx = self.max()
    if mx - mn == 0:
        return SmartArray(np.zeros_like(self, dtype=float))
    result = (self - mn) / (mx - mn)
    return SmartArray(result)
```

6.4 Overriding __str__() - Polymorphism

By overriding `__str__()` we show polymorphism in our own code. The same `print()` call produces completely different output depending on whether the object is a plain `ndarray` or our `SmartArray`. The parent `ndarray` just prints the numbers. Our `SmartArray` also shows the shape and dtype as a label on top.

```
def __str__(self):
    array_string = np.array_str(np.asarray(self))
    return 'SmartArray(shape=' + str(self.shape) + \
    ', dtype=' + str(self.dtype) + ')\n' + array_string

normal = np.array([1, 2, 3])
smart = SmartArray([1, 2, 3])

print(normal) # [1 2 3]
print(smart) # SmartArray(shape=(3,), dtype=int64)
# [1 2 3]
```

6.5 The is_symmetric() Method - Inheritance

The `is_symmetric()` method checks whether a 2D array is equal to its own transpose. A symmetric matrix is one where $a[i][j]$ equals $a[j][i]$ for every position. This is useful in linear algebra where certain algorithms require the input matrix to be symmetric. The method uses `self.T` to get the transpose, which we get for free from `ndarray`, and `np.array_equal` to compare the two.

```
def is_symmetric(self):
    if self.ndim != 2:
        return False
    return np.array_equal(self, self.T)

sym = SmartArray([[1, 2, 3], [2, 5, 6], [3, 6, 9]])
```

```
print(sym.is_symmetric()) # True
not_sym = SmartArray([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(not_sym.is_symmetric()) # False
```

8. Conclusion

This project showed us that real-world, professional libraries are not built by randomly putting code together. NumPy is a great example of how OOP principles can be used thoughtfully to make something that is both powerful and easy to use at the same time.

Encapsulation is at work the moment you create an array. All the messy byte-level details are hidden and you get clean attributes instead. Inheritance lets the matrix, memmap, and MaskedArray classes each build on top of ndarray without rewriting anything from scratch. Polymorphism means the `+` operator and the `.dot()` method can produce completely different results depending on their input. And abstraction lets you call `.reshape()` or `.astype()` without ever thinking about what is happening at the memory level.

Building SmartArray was the part that made these ideas really stick for us. When we had to use `__new__` instead of `__init__` just because NumPy works at the C level, or when we had to think about what `__array_finalize__` does and why it needs to be there even when empty, we understood that inheritance is not always simple. Sometimes the parent class has its own rules that you have to follow carefully.

The design decision about homogeneous types also taught us something important. Sometimes a strict constraint is actually a feature and not a limitation. NumPy's homogeneity is what makes it so fast. And the fact that Pandas was built on top of it to handle the cases NumPy cannot deal with is a good example of how layers of software should work. Each tool does what it is best at.

Overall this project gave us a much better understanding of OOP than just reading definitions from a textbook. Seeing these principles in actual code that millions of people use every day made everything much more clear and meaningful.